HDOC Day-71: Dynamic Memory Allocation and Matrix Operations in C

Aim

To dynamically allocate two matrices and perform addition, subtraction, and multiplication using pointer notation and separate user-defined functions. All allocated memory is released properly.

Algorithm

1. Read the dimensions of both matrices and validate that they are positive.

2. Allocate memory for matrices A and B using malloc(), and verify allocation success.

3. Read and display both matrices using pointer arithmetic.

4. If dimensions match, add corresponding elements in a separately allocated result matrix, display it, and free it.

5. If dimensions match, subtract corresponding elements in a separately allocated result matrix, display it, and free it.

6. If the columns of A equal the rows of B, multiply using three nested loops, display the result, and free its memory.

7. Display messages for incompatible dimensions and release matrices A and B using free().

C Program

```
#include <stdio.h>
#include <stdlib.h>

void readMatrix(int *mat, int rows, int cols);
void displayMatrix(int *mat, int rows, int cols);
void addMatrices(int *A, int *B, int r, int c);
void subtractMatrices(int *A, int *B, int r, int c);
void multiplyMatrices(int *A, int *B, int r1, int c1, int c2);

int main()
{
    int r1, c1, r2, c2;
    int *A, *B;

    printf("Enter rows and columns of Matrix A: ");
    scanf("%d %d", &r1, &c1);
    printf("Enter rows and columns of Matrix B: ");
    scanf("%d %d", &r2, &c2);

    if (r1 <= 0 || c1 <= 0 || r2 <= 0 || c2 <= 0)
    {
        printf("Invalid matrix dimensions!\n");
        return 0;
    }

    A = (int *)malloc(r1 * c1 * sizeof(int));
    B = (int *)malloc(r2 * c2 * sizeof(int));
    if (A == NULL || B == NULL)
    {
        printf("Memory allocation failed!\n");
        free(A);
        free(B);
        return 0;
    }

    printf("\nEnter elements of Matrix A:\n");
    readMatrix(A, r1, c1);
    printf("\nEnter elements of Matrix B:\n");
```

```
    readMatrix(B, r2, c2);

    printf("\nMatrix A:\n");
    displayMatrix(A, r1, c1);
    printf("\nMatrix B:\n");
    displayMatrix(B, r2, c2);

    printf("\n----- MATRIX ADDITION -----\n");
    if (r1 == r2 && c1 == c2)
        addMatrices(A, B, r1, c1);
    else
        printf("Addition not possible!\n");

    printf("\n----- MATRIX SUBTRACTION -----\n");
    if (r1 == r2 && c1 == c2)
        subtractMatrices(A, B, r1, c1);
    else
        printf("Subtraction not possible!\n");

    printf("\n----- MATRIX MULTIPLICATION -----\n");
    if (c1 == r2)
        multiplyMatrices(A, B, r1, c1, c2);
    else
        printf("Multiplication not possible!\n");

    free(A);
    free(B);
    printf("\nMemory released successfully.\n");
    return 0;
}

void readMatrix(int *mat, int rows, int cols)
{
    int i, j;
    for (i = 0; i < rows; i++)
        for (j = 0; j < cols; j++)
            scanf("%d", (mat + i * cols + j));
}

void displayMatrix(int *mat, int rows, int cols)
{
    int i, j;
    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < cols; j++)
            printf("%d ", *(mat + i * cols + j));
        printf("\n");
    }
}

void addMatrices(int *A, int *B, int r, int c)
{
    int i, j;
    int *C = (int *)malloc(r * c * sizeof(int));
    if (C == NULL)
    {
        printf("Memory allocation failed!\n");
        return;
    }
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            *(C + i * c + j) = *(A + i * c + j)
                             + *(B + i * c + j);
    printf("Addition Result:\n");
    displayMatrix(C, r, c);
    free(C);
}

void subtractMatrices(int *A, int *B, int r, int c)
{
    int i, j;
```

```
    int *C = (int *)malloc(r * c * sizeof(int));
    if (C == NULL)
    {
        printf("Memory allocation failed!\n");
        return;
    }
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            *(C + i * c + j) = *(A + i * c + j)
                               - *(B + i * c + j);
    printf("Subtraction Result:\n");
    displayMatrix(C, r, c);
    free(C);
}

void multiplyMatrices(int *A, int *B,
                      int r1, int c1, int c2)
{
    int i, j, k;
    int *C = (int *)calloc(r1 * c2, sizeof(int));
    if (C == NULL)
    {
        printf("Memory allocation failed!\n");
        return;
    }
    for (i = 0; i < r1; i++)
        for (j = 0; j < c2; j++)
            for (k = 0; k < c1; k++)
                *(C + i * c2 + j) +=
                    *(A + i * c1 + k) * *(B + k * c2 + j);
    printf("Multiplication Result:\n");
    displayMatrix(C, r1, c2);
    free(C);
}
```

Explanation

Dynamic allocation: malloc() reserves memory for A and B, while calloc() initializes the multiplication result to zero. Each allocated block is released with free().

Pointer notation: *(A + i * cols + j) accesses the element in row i, column j of a contiguous matrix allocation.

Addition and subtraction: Both matrices must have identical dimensions. The corresponding elements are added or subtracted.

Multiplication: The number of columns in A must equal the number of rows in B. Each result entry is the sum of products across the corresponding row and column.

Worked Example

```
Matrix A:       Matrix B:
1 2             5 6
3 4             7 8

A + B:          A - B:          A x B:
6 8             -4 -4           19 22
10 12           -4 -4           43 50
```

Test Case Table

Case	Input description	Expected result
1	Two 2 x 2 matrices	All three operations succeed
2	Two 3 x 3 matrices	All three operations succeed

Case	Input description	Expected result
3	A is identity matrix	A x B equals B
4	B is zero matrix	A x B is zero matrix
5	Negative elements	Signed results calculated
6	Unequal dimensions	Addition and subtraction rejected
7	A: 2 x 3, B: 3 x 2	Multiplication succeeds
8	A: 2 x 3, B: 2 x 2	Multiplication rejected
9	Zero dimension	Invalid dimensions
10	Negative dimension	Invalid dimensions

Sample Output - Equal Dimensions

```
Enter rows and columns of Matrix A: 2 2
Enter rows and columns of Matrix B: 2 2
Enter elements of Matrix A:
1 2
3 4
Enter elements of Matrix B:
5 6
7 8

Matrix A:
1 2
3 4
Matrix B:
5 6
7 8

----- MATRIX ADDITION -----
Addition Result:
6 8
10 12

----- MATRIX SUBTRACTION -----
Subtraction Result:
-4 -4
-4 -4

----- MATRIX MULTIPLICATION -----
Multiplication Result:
19 22
43 50

Memory released successfully.
```

Sample Output - Different Dimensions

```
Enter rows and columns of Matrix A: 2 3
Enter rows and columns of Matrix B: 3 2
Enter elements of Matrix A:
1 2 3
4 5 6
Enter elements of Matrix B:
7 8
9 10
11 12

----- MATRIX ADDITION -----
```

```
Addition not possible!
----- MATRIX SUBTRACTION -----
Subtraction not possible!
----- MATRIX MULTIPLICATION -----
Multiplication Result:
58 64
139 154
Memory released successfully.
```

Sample Output - Invalid Dimensions

```
Enter rows and columns of Matrix A: 0 2
Enter rows and columns of Matrix B: 2 2
Invalid matrix dimensions!
```

Result

The program demonstrates dynamic memory allocation and pointer-based matrix access. It performs addition, subtraction, and multiplication in separate functions, checks dimensional compatibility, displays results, and frees allocated memory.

Note: The assignment code assumes valid integer input and matrix sizes that do not overflow allocation-size or integer arithmetic calculations.